VSEPR-SIM Version Progression — Broad Methodology Update

v3.0.0 → v4.0.0 → v5.0.0 | Native VSIM Architecture | Active v5 Beta Series

Purpose

This document updates the original v3.0.0 Formation Engine methodology summary into the current v5.0.0 project framing. The original v3.0.0 methodology described a classical atomistic formation engine with advanced but mostly disconnected features. The current v5.0.0 architecture reframes those features as part of a native, deterministic, multiscale simulation and analysis environment.

The major progression is:

advanced features → connected workflows → native multiscale language + runtime platform

v3.0.0 — Advanced Feature Introduction

Version 3.0.0 was the first stage where VSEPR-SIM moved beyond simple structure generation and began introducing advanced simulation concepts. It included early support for atomistic formation, force evaluation, thermodynamic bookkeeping, FIRE minimisation, charge models, scoring, self-audit tools, and early multiscale placeholders.

Its value was conceptual expansion. It showed that the project could support a larger scientific architecture: molecular formation, classical MD, energy accounting, statistical convergence, reproducibility, and future coarse-grained simulation.

Its limitation was architectural disconnection. Many components existed as promising modules, but they were not yet unified by a common runtime, common scripting language, common output doctrine, or material-property inference pipeline.

v3.0.0 summary: advanced concepts existed, but the system was still a collection of partially connected methods rather than a complete platform.

v4.0.0 — Workflow Connection and Expansion

Version 4.0.0 connected more of the v3.0.0 ideas into broader workflows. Structure generation, simulation, analysis, reporting, and automation became more directly linked. This version represented the first serious attempt to turn isolated scientific modules into an operational simulation environment.

The v4.x line also expanded the project's ambition. It experimented with larger workflows, wrapper behavior, automation, external tooling, and broader usability. Later v4.4-era development introduced a Python library/wrapper phase, which improved experimentation and scripting convenience.

However, late v4.x became too Python-centered. Python was useful for testing, plotting, and quick workflows, but it also pulled too much project identity outside the native simulator. Core simulation behavior, orchestration, and analysis needed to belong to VSEPR-SIM itself rather than to an external wrapper layer.

v4.0.0 summary: the system became more connected, but late v4.x began depending too heavily on Python-centered workflows.

v5.0.0 — Native VSIM Architecture

Version 5.0.0 introduces the native VSIM scripting language and reorganizes VSEPR-SIM around deterministic simulation control, explicit file doctrine, material-property inference, and multiscale analysis.

Unlike v3.0.0, v5.0.0 does not treat advanced features as isolated additions. Unlike late v4.x, it does not place Python at the center of the workflow. The core direction is now native: scripts, runs, outputs, analysis, verification, and reporting are controlled through VSEPR-SIM's own language and runtime.

The v5.0.0 line has developed through an active beta series. Across these iterations, the system has moved toward:

- native `.vsim` scripting and runtime control;
- removal of Python-dependent core modules;
- deterministic `.xyz`, `.xyzf`, and `.xyzFull` output doctrine;
- state/history files as ground truth rather than containers for inferred labels;
- post-run analysis for structure, diffusion, packing, defects, surfaces, and transport;
- material-property inference from trajectories and sampled behavior;
- reproducible reports, verification gates, and structured artifact generation.

v5.0.0 summary: v5 converts the disconnected v3 concepts and the Python-heavy v4 workflows into a native, deterministic, multiscale simulation architecture.

Updated State Doctrine

The original v3.0.0 methodology described a canonical atomistic state with identity, phase, and scratch partitions. In v5.0.0 this doctrine is retained but clarified: state files must store ground-truth simulation data, not final interpretations.

The updated rule is:

$$\text{state/history truth} \neq \text{derived material meaning}.$$

Files such as `.xyz`, `.xyzf`, and `.xyzFull` should store positions, identities, frame history, velocities/orientations where needed, and energy or runtime traces. Material labels such as defect class, transport regime, packing state, diffusion class, or macro-readiness should be computed afterward by the analysis layer.

This separation is central to the v5 architecture because it prevents the simulator from hardcoding the answer it is supposed to measure.

Updated Simulation Pipeline

The v3 pipeline centered on formation and force evaluation. The v5 pipeline is broader:

`.vsim script` → runtime → state/trajectory → analysis → inferenc

A v5 run may include initialization, formation, relaxation,

trajectory capture, structural analysis, property sampling, verification gates, and report generation. The important change is that these are no longer treated as loose utilities. They are encoded as part of a scripted workflow.

Material-Property Inference

The defining v5 expansion is the shift from structure generation toward material-property inference. Instead of assigning properties directly, VSEPR-SIM measures behavior from simulation output and infers higher-level properties from those measurements.

Examples include:

- diffusion behavior from mean-square displacement and trajectory spread;
- packing behavior from porosity, coordination, and compression response;
- defect behavior from residuals, identity mismatch, vacancies, substitutions, and interstitials;
- surface behavior from contact, residence, embedding, and anisotropic motion;
- macro-readiness from field projection, RVE sampling, mass conservation, and emergence metrics.

This converts the original v3 advanced-feature set into a practical multiscale analysis platform.

Python Dependency Removal

The v3/v4 documents referenced Python tools for classification, gap targeting, regression detection, plotting, and workflow support. In v5.0.0 these tools should be treated as historical or optional development aids, not core system requirements.

The updated requirement is that the core simulator must remain native, reproducible, and independent of Python execution. Python may still be useful for external plotting, temporary analysis, or legacy compatibility, but it is not the governing architecture.

Version Comparison

Version	Main Role	Limitation Solved by Later Work
v3.0.0	Introduced advanced formation, force, thermodynamic, charge, scoring, and multiscale concepts.	Advanced features were disconnected and lacked a unified native workflow.
v4.0.0	Connected modules into broader workflows and expanded ambition.	Late v4.x became too Python-wrapper focused.
v5.0.0	Rebuilt the project around VSIM scripting, deterministic files, analysis gates, and material inference.	Still active; architecture is maturing across beta iterations.

Current v5 Framing

VSEPR-SIM v5.0.0 should be presented as the first mature architectural consolidation of the project. Earlier versions remain important, but their role changes in the documentation.

v3.0.0 should be described as the first advanced feature foundation. v4.0.0 should be described as the first integration and workflow expansion. v5.0.0 should be described as the native language and multiscale inference architecture that makes the earlier concepts usable as a coherent research platform.

Updated from the original v3.0.0 Formation Engine methodology summary. This document is intended as a broad transition note for rewriting legacy v3/v4 documentation into the v5.0.0 VSIM architecture.